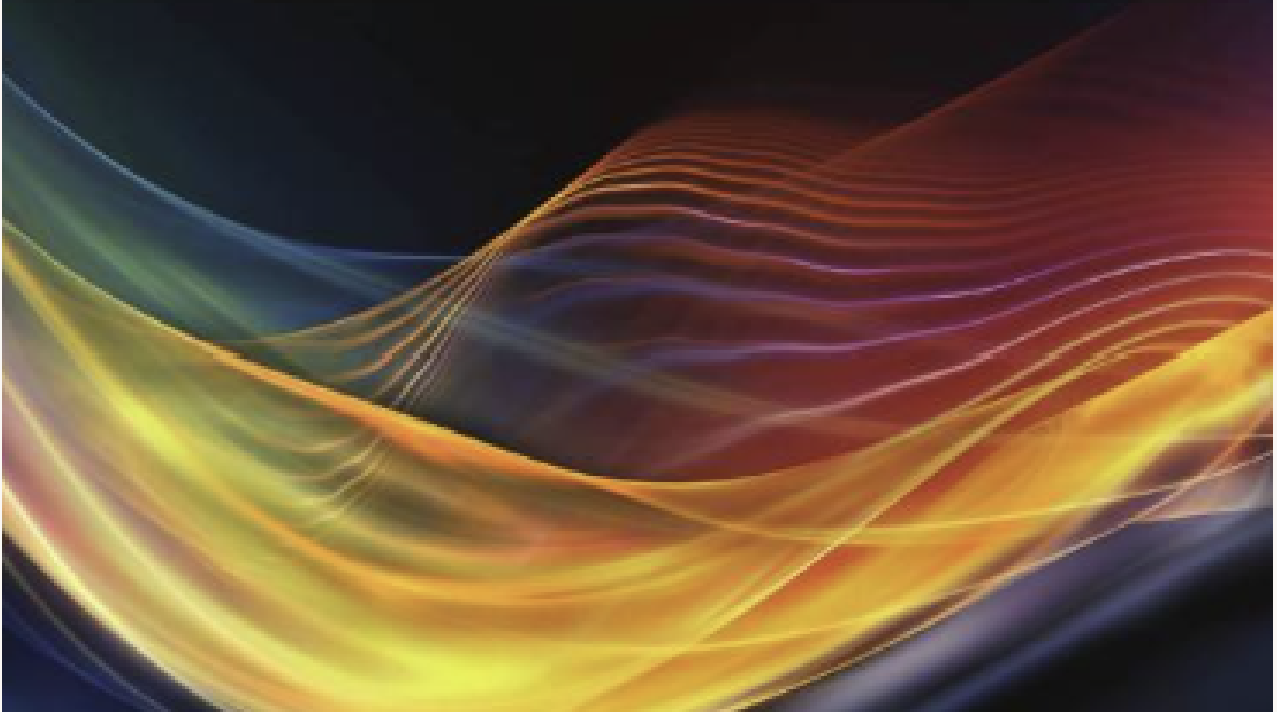




PROGRAMMATION PYTHON - MGA802

Mini-projet B : Analyse et Intégration numérique

Session Été 2026 — Responsable : Ilyass Tabiai

ÉQUIPE : BLANCHARD Flavien, Milissa MECHREF, Vincent CONDETTE

CODE PERMANENT : BLAF63330301, MECM76290102, CONV76330301

1 Répartition des tâches

Pour mener à bien ce projet, l'équipe s'est répartie les différentes tâches :

- **Vincent CONDETTE** : Rédaction du rapport (synthèse des performances, ordres de convergence et erreurs) et rédaction du fichier `README.md`.
- **Milissa MECHREF** : Revue des commentaires, vérification et validation de la qualité et de la robustesse du code pour les deux modules.
- **Flavien BLANCHARD** : Création du code (`methode_integration.py`, incluant les fonctions d'intégration numérique avec les différentes approches (itératives, vectorisées et pré-programmées). Configuration du script principal `analyse_numerique.py`.

2 Objectif du projet

Ce projet vise à comparer l'efficacité des approches par vectorisation avec NumPy et avec l'utilisation de méthodes pré-programmées dans des paquets python (SciPy) face aux implémentations itératives en Python standard.

3 Architecture du projet

Afin de répondre aux exigences du projet, l'architecture du programme se divise en deux modules. Cette approche permet de séparer la logique mathématique de la génération des résultats graphiques.

Le module d'intégration (`methode_integration.py`) agit comme une bibliothèque numérique indépendante. Il inclut la définition de la fonction à intégrer et les algorithmes d'intégration (méthodes des rectangles, des trapèzes et de Simpson). Pour permettre une analyse comparative, chaque algorithme d'intégration est effectué avec plusieurs implémentations (itératives classiques, vectorisées avec NumPy et pré-programmées via SciPy). Il sera alors possible de comparer le temps de calcul de chaque approche.

Le script d'exécution principal (`analyse_numerique.py`) importe le module d'intégration, définit les coefficients du polynôme, appelle les fonctions et effectue les exécutions. Il calcule alors les erreurs absolues et génère les graphiques comparatifs.

4 Principes des méthodes d'intégration

L'objectif est de calculer l'aire sous la courbe d'une fonction polynomiale du 3^e ordre de la forme $f(x) = p_0 + p_1x + p_2x^2 + p_3x^3$ sur un intervalle $[a, b]$ divisé en n segments de largeur h . Les trois stratégies d'approximation géométrique sont les suivantes :

- **Rectangles** : Évaluer la fonction au bord gauche de chaque sous-intervalle. L'aire globale est la somme de rectangles de hauteur $f(x_i)$. L'algorithme est simple mais a une convergence lente d'ordre $\mathcal{O}(h^1)$.
- **Trapèzes** : Interpoler la fonction par un segment de droite reliant les bornes de chaque intervalle. Cette approche améliore la précision et offre une convergence d'ordre $\mathcal{O}(h^2)$.
- **Simpson** : Approcher la fonction sur chaque segment par une parabole passant par les extrémités et le point milieu, avec une pondération $(1, 4, 1)$. Sa convergence est d'ordre $\mathcal{O}(h^4)$ et elle est théoriquement exacte pour les polynômes de degré ≤ 3 .

Chaque méthode a été développée selon trois approches d'implémentation distinctes :

- **Approche Classique** : Utilisation de boucles `for` en Python (lisible, mais vitesse d'exécution limitée).
- **Approche Vectorisée (NumPy)** : Évaluation simultanée via des tableaux de données, remplaçant les boucles par des méthodes optimisées en C.
- **Approche Pré-programmée (SciPy)** : Utilisation des fonctions natives de `scipy.integrate` pour valider la précision et servir de référence de performance.

5 Étude comparative et analyse des performances

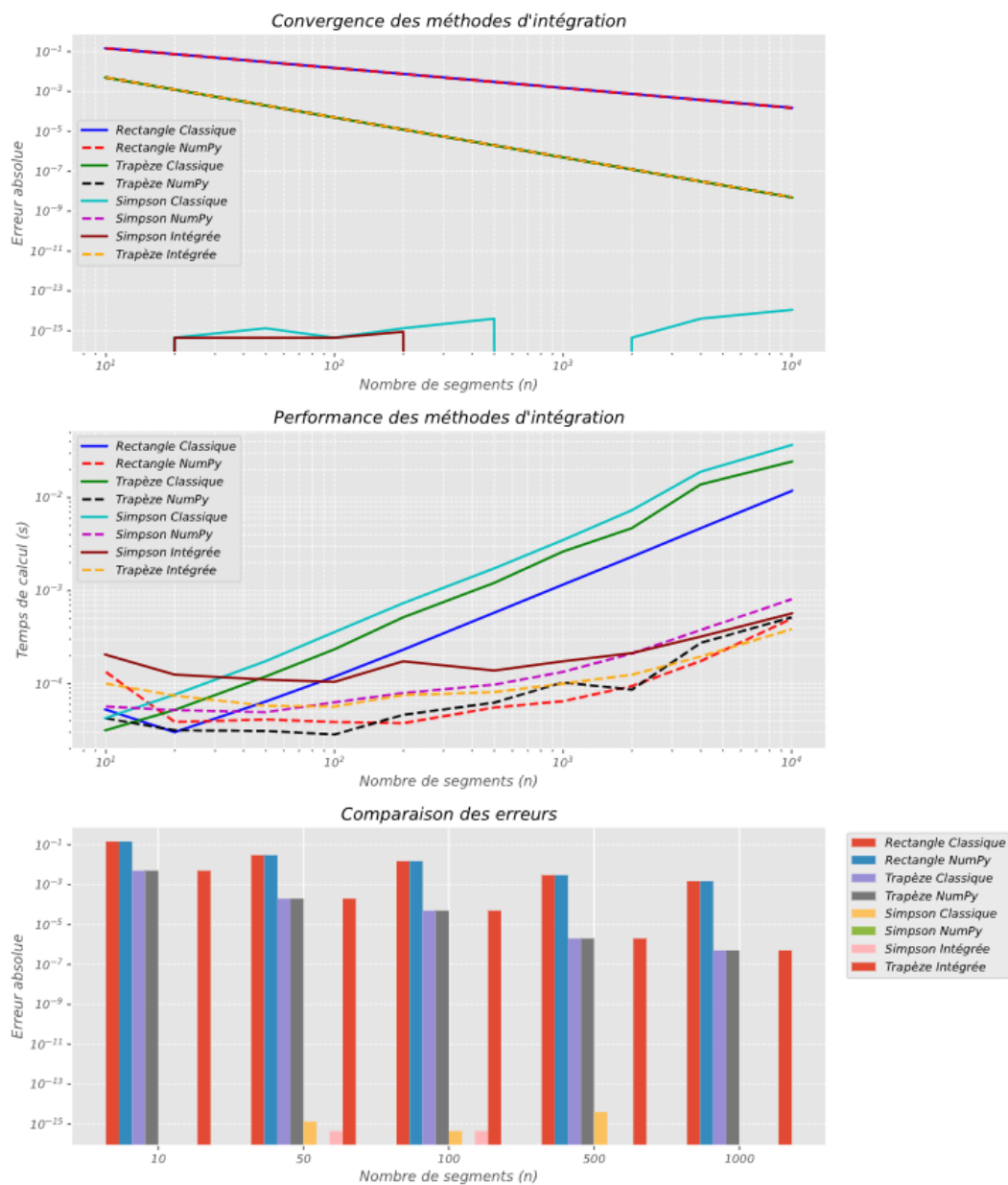


FIGURE 1 – Évaluation comparative des méthodes : convergence théorique, temps d'exécution et comparaison des erreurs absolues en fonction du nombre de segments.

5.1 Analyse de la convergence

- **Méthode des rectangles** : L'erreur décroît de manière proportionnelle à l'inverse du nombre de segments. La pente observée de -1 valide la convergence globale d'ordre $\mathcal{O}(h^1)$.
- **Méthode des trapèzes** : L'interpolation linéaire sur chaque segment permet une chute plus rapide de l'erreur. La pente de -2 confirme l'ordre de convergence de $\mathcal{O}(h^2)$.
- **Méthode de Simpson** : La fonction intégrée étant un polynôme du 3^e ordre, l'approximation quadratique de Simpson est mathématiquement exacte. Le résidu observé de l'ordre de 10^{-15} ne correspond qu'à la limite de précision machine.

La méthode d'implémentation (Classique, NumPy, SciPy) n'influence pas l'erreur d'approximation. On observe bien une superposition parfaite de courbes de convergence respectives.

5.2 Impact des méthodes d'implémentations sur le temps de calcul

Pour les implémentations « classiques » (boucles for), le temps de calcul croît de manière linéaire avec le nombre de segments. Pour $n = 10^4$, ces méthodes nécessitent un temps de l'ordre de 10^{-2} seconde car l'exécution se fait ligne par ligne à chaque répétition de la boucle.

L'utilisation de la bibliothèque **NumPy** assure une évolution asymptotique du temps de calcul. Pour des valeurs de n élevées ($n > 100$), les méthodes vectorisées s'exécutent bien plus rapidement que les méthodes classiques. En effet, la suppression des boucles **for** pour des opérations sur des tableaux en C, permet de réduire fortement le temps de calcul (moins de 10^{-3} seconde pour $n = 10^4$).

Les méthodes pré-programmées de **SciPy** montrent aussi de bonnes performances, similaires à celles des implémentations NumPy. Cela s'explique par le fait que ces routines optimisent le temps de calcul en déléguant les opérations mathématiques à des langages plus efficaces comme le C ou Fortran.

5.3 Comparaison et distribution des erreurs

- Pour une faible discrétisation ($n = 10$), la méthode des rectangles affiche une erreur importante de l'ordre de 10^{-1} . La méthode des trapèzes est plus précise, avec une erreur de l'ordre de 10^{-2} . En revanche, la méthode de Simpson présente directement une erreur négligeable de 10^{-15} . Cela démontre l'efficacité d'une approximation d'ordre supérieur, même sur un maillage très approximatif.
- Pour une discrétisation importante ($n = 1000$), l'écart entre les méthodes géométriques s'accroît. La méthode des trapèzes est plus précise (10^{-6}), tandis que la méthode des rectangles stagne à 10^{-3} , ce qui confirme son inefficacité pour des calculs de haute précision. On remarque aussi que l'erreur de la méthode de Simpson ne dépend pas de n pour un polynôme d'ordre 3. L'erreur ne représente que la limite de la précision machine, liée aux variables float.

5.4 Conclusion

Le choix de la méthode d'intégration impacte fortement la précision de la solution, tandis que le choix de l'implémentation algorithmique (vectorisation avec NumPy ou intégration via SciPy) assure l'efficacité et la rapidité du modèle pour des maillages de grande dimension.